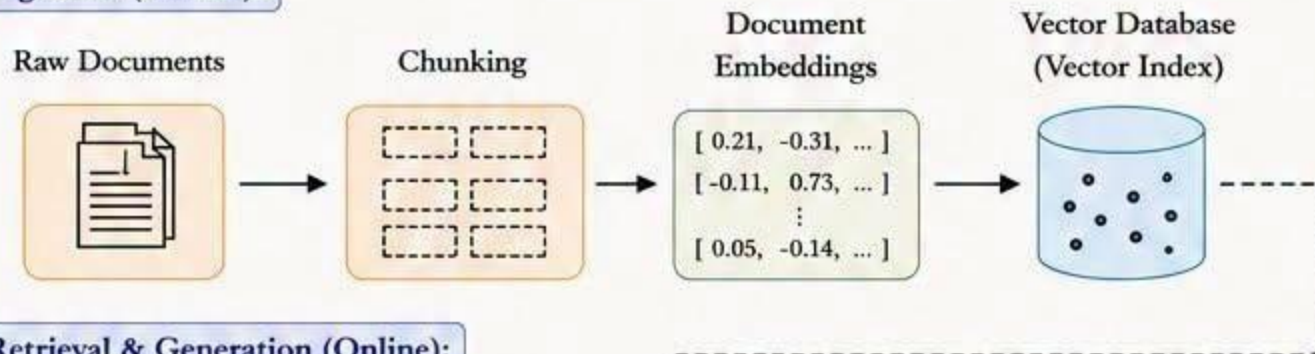
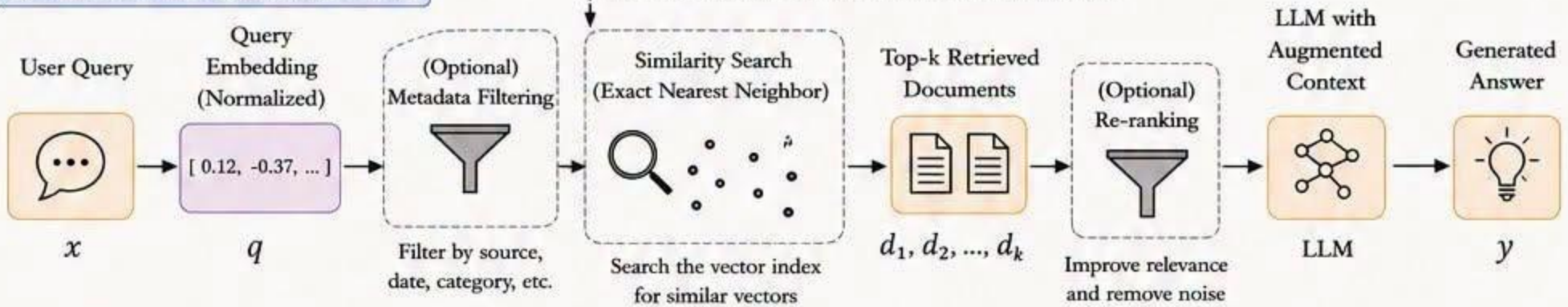


Retrieval-Augmented Generation (RAG)

1) Ingestion (Offline):



2) Retrieval & Generation (Online):



- **Goal:** Enhance LLM responses by retrieving relevant external knowledge.

- **Pipeline:**

1. **Ingestion (offline):** Load documents, split into chunks, create embeddings, and store in a vector database.
2. **(Optional)** Apply metadata filtering to limit the search space.
3. **Retrieve (online):** Convert the user query to an embedding and perform similarity search to get top-k relevant documents.
4. **(Optional)** Re-rank results to improve relevance.
5. Provide the retrieved context and query to the LLM.
6. Generate the final answer grounded in the provided context.

- **Key Components:**

- **Retriever:** Performs similarity search over the vector database.
- **Vector Database:** Stores document embeddings for efficient retrieval.
- **LLM (Generator):** Produces answers using both the query and retrieved context.

- **Benefits:** Reduces hallucinations, improves factual accuracy, and keeps knowledge up to date.



Credit: MLTut

RAG: Python Code Example

```
# 1. Ingestion: Load documents, chunk, embed, and build vector index
import numpy as np
import faiss
from sentence_transformers import SentenceTransformer

docs = ["RAG combines retrieval and generation.",
        "It retrieves relevant documents from a vector store.",
        "The LLM uses the retrieved context to generate answers."]

model = SentenceTransformer('all-MiniLM-L6-v2')

# Create embeddings for documents
doc_embeddings = model.encode(docs, convert_to_numpy=True)

# Normalize document embeddings for cosine similarity (required for IndexFlatIP)
doc_embeddings = doc_embeddings.astype('float32')
faiss.normalize_L2(doc_embeddings)

# Build FAISS index (cosine similarity via inner product on normalized vectors)
dimension = doc_embeddings.shape[1]
index = faiss.IndexFlatIP(dimension) # Use IndexFlatIP for cosine similarity
index.add(doc_embeddings)

# 2. Define the RAG pipeline
def rag_answer(query, top_k=2):
    # Embed and normalize the query
    q_emb = model.encode([query], convert_to_numpy=True)
    q_emb = q_emb.astype('float32')
    faiss.normalize_L2(q_emb)

    # Similarity search (retrieve top-k)
    D, I = index.search(q_emb, top_k)

    # Gather retrieved documents
    retrieved_docs = [docs[i] for i in I[0]]
    context = "\n\n".join(retrieved_docs)

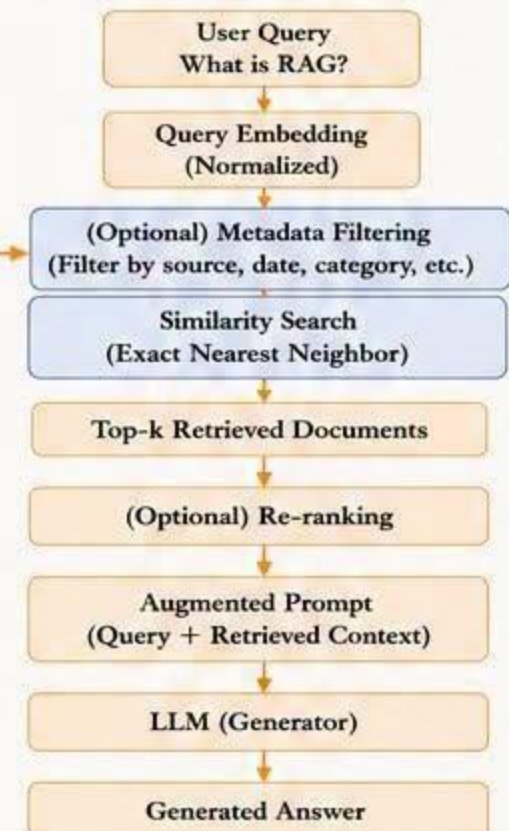
    # Build prompt
    prompt = f"Use the following context to answer the question.\n\nContext: {context}\n\nQuestion: {query}"

    # Generate answer
    outputs = generator(prompt, max_new_tokens=200, do_sample=False)
    generated = outputs[0]['generated_text']

    # Robust extraction: remove the prompt and strip whitespace
    answer = generated[len(prompt):].strip()

    return answer
```

```
# 3. Example
query = "What is RAG?"
answer = rag_answer(query)
print(f"Question: {query}")
print(f"Answer: \n{answer}")
```



RAG combines retrieval and generation:
Answer = LLM(Query + Retrieved Context)

Credit: MLTut

RAG VS AGENTIC RAG

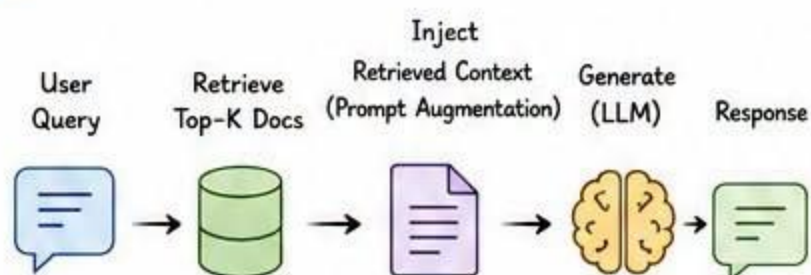
From single-step retrieval to autonomous reasoning and action.

RAG (Retrieval-Augmented Generation)

What it is

Enhances LLM responses by retrieving relevant information from external sources at query time.

Workflow



Key Characteristics

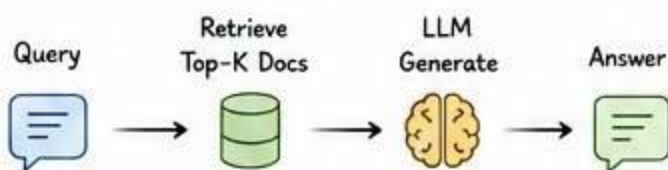
- Single-pass retrieval
- Static context
- No iterative planning or execution
- Best for straightforward Q&A
- Limited to information lookup

Use Cases

- FAQ systems
- Document Q&A
- Knowledge base search
- Summarization

Example Scenario

Q: What are the benefits of renewable energy?



Single retrieval phase → Single response generation

Pros & Cons

- ✓ **Pros**
 - Simple to implement
 - Efficient and fast
 - Fewer resources required
- ✗ **Cons**
 - Limited to single-step reasoning
 - May miss deeper insights
 - No adaptability

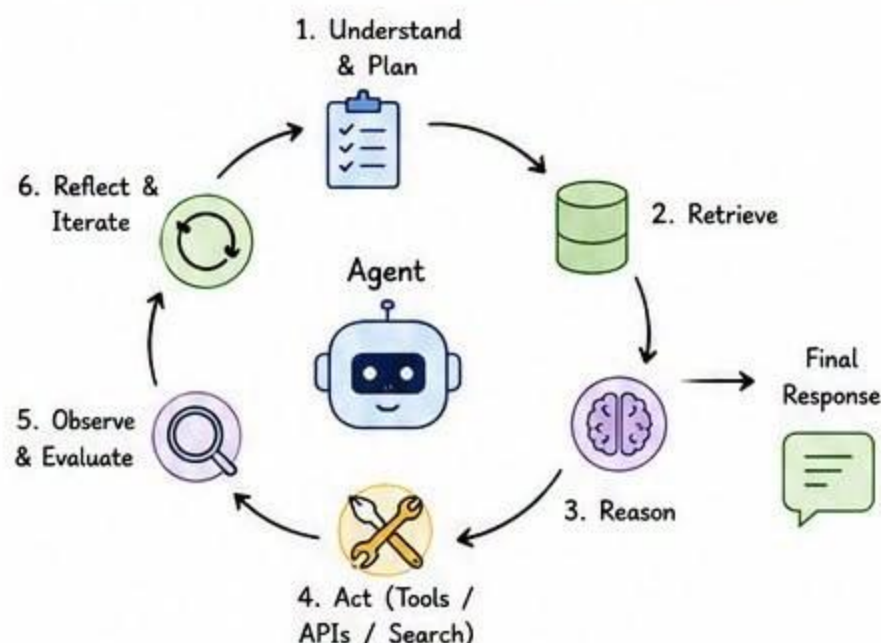
Key Difference (Summary)

Retrieve → Inject Retrieved Context → Generate
One-shot, passive retrieval

AGENTIC RAG (Agentic Retrieval-Augmented Generation)

(Agentic Retrieval-Augmented Generation)

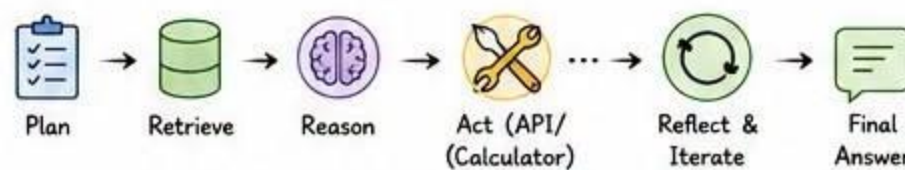
Uses an autonomous agent to iteratively plan, retrieve information, reason, invoke tools, and refine responses for complex tasks.



- Iterative reasoning and retrieval
- Stateful execution with working memory
- Can invoke tools, APIs, databases, search engines, and external services
- Adapts and refines based on feedback
- Best for complex, multi-hop tasks

- Research and analysis
- Data exploration
- Multi-hop reasoning tasks
- Tool-augmented automation
- Decision making & planning

Q: Compare solar and wind energy, analyze recent trends, and suggest the best option for a small city.



Iterative reasoning → Refined final response

- ✓ **Pros**
 - Handles complex tasks
 - Adaptive and iterative
 - Can use tools and take actions
- ✗ **Cons**
 - More complex to build
 - Higher cost and latency
 - Risk of errors in multi-step reasoning

Plan → Retrieve → Reason → Act → Observe → Reflect → Iterate
Multi-step, active and autonomous

Key Takeaway

RAG retrieves and answers.
Agentic RAG thinks, acts, and solves.



Choose RAG for simple lookups.



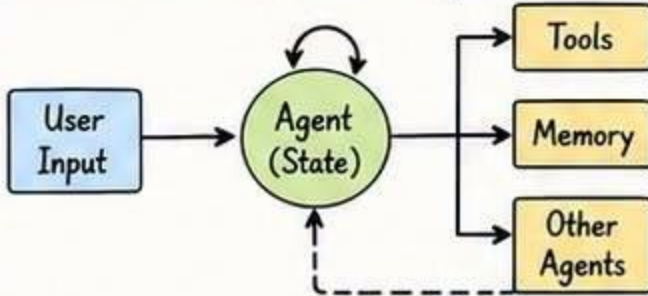
Choose Agentic RAG for complex, real-world tasks.

AI Agent Frameworks

Popular Frameworks for Building AI Agents

1. LangGraph

Build stateful, multi-actor agents with control flow and memory.



Key Features

- Graph-based workflow & control flow
- State management & memory
- Multi-agent collaboration
- Human-in-the-loop support
- Streaming & async support

Best For

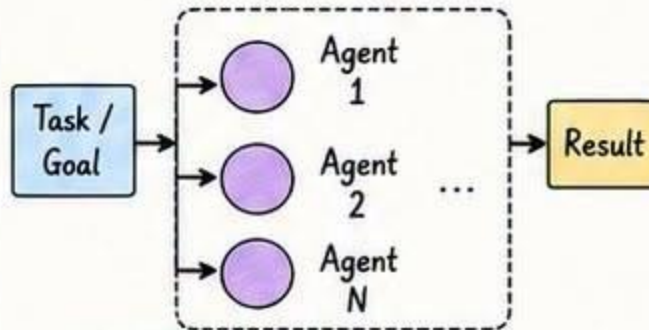
Complex, long-running, stateful agent applications

Tech Stack



2. CrewAI

Build role-playing AI agents that work together as a crew.



Key Features

- Role-based agents with specific goals
- Task delegation & collaboration
- Process management
- Memory & context handling
- Integrations with many tools

Best For

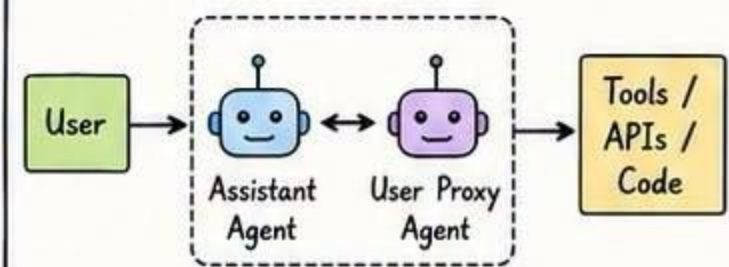
Team of agents working on complex tasks together

Tech Stack



3. AutoGen

Enable LLMs to converse, plan, and solve tasks autonomously.



Key Features

- Multi-agent conversation
- Code execution & tool use
- Human-in-the-loop
- Flexible agent types
- Supports complex workflows

Best For

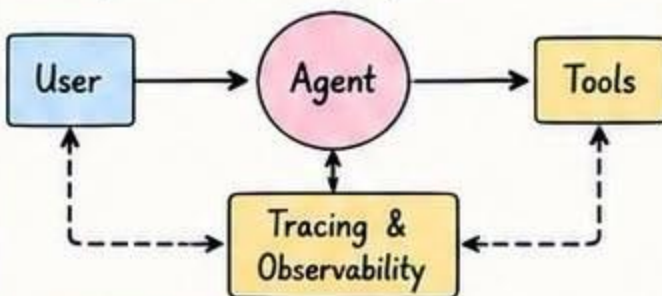
Conversational agents, coding agents, research agents

Tech Stack



4. OpenAI Agents SDK

Official framework by OpenAI to build production-ready agents.



Key Features

- Built by OpenAI
- Integrated with OpenAI models & tools
- Tracing, evaluation & observability
- Guardrails & safety
- Production-ready primitives

Best For

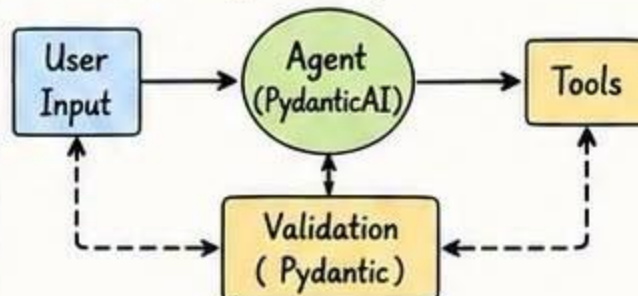
Production-grade agents with reliability & observability

Tech Stack



5. PydanticAI

Build reliable AI agents with data validation & type safety.



Key Features

- Data validation with Pydantic
- Structured outputs
- Type-safe agent development
- Easy tool integration
- Lightweight & developer friendly

Best For

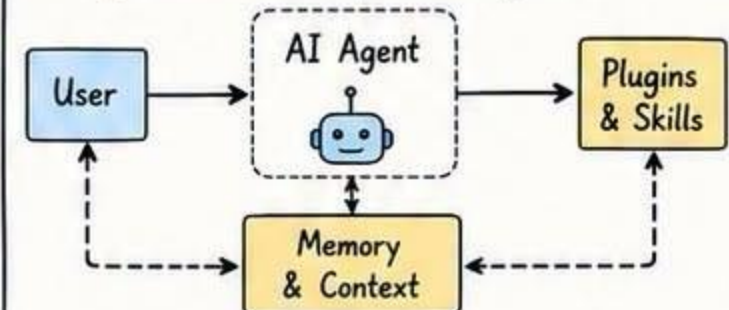
Reliable, structured, and type-safe agent applications

Tech Stack



6. Semantic Kernel

Microsoft's framework to build AI agents with enterprise integrations.



Key Features

- Enterprise ready
- Plugins (Skills) & Connectors
- Memory & Context management
- Multi-language support (Python, C#, Java)
- Integrates with Azure services

Best For

Enterprise & Microsoft ecosystem based agent solutions

Tech Stack

